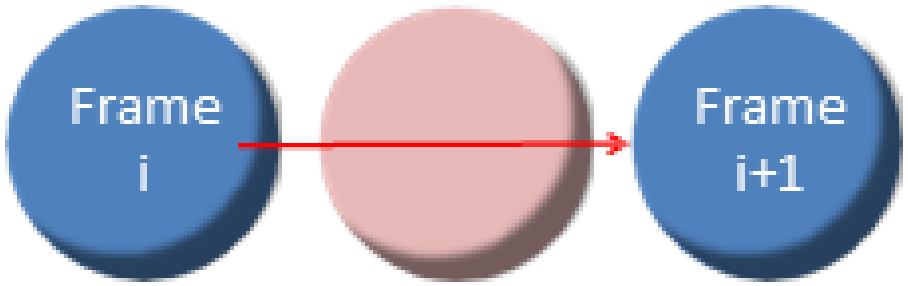
Collisions

- Gongbo (Tony) Liang, PhD

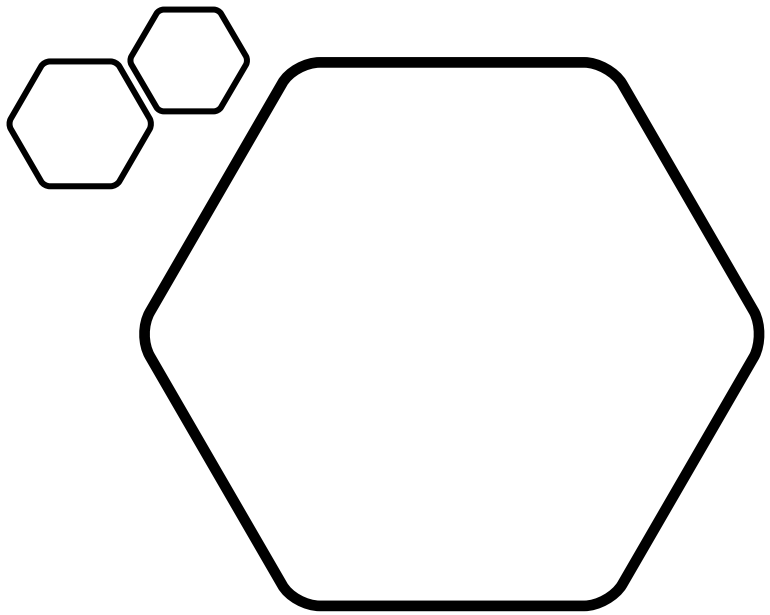
TEXAS A&M UNIVERSITY
SAN ANTONIO

Collisions



- Detect collisions
  - respond to these collisions
- Happens in two ways:
  - Simply trying to limit motion
    - i.e., not walking through walls
      - Prevent the update function from continuing to increase the position values for that axis
  - Simulate a physical reaction
    - Laws of physics are applied to determine how to respond to a collision.

Detect Collisions

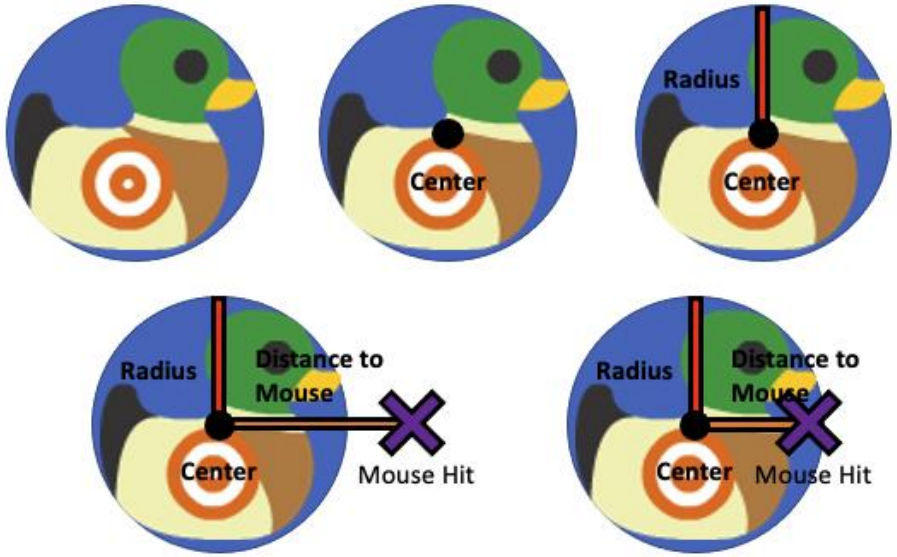


- Lots of ways to detect collisions
- **Most obvious method**
  - Take every single vertex in a model and check to see if any of them are inside another model
  - Very time-consuming, especially if we have lots of models, or models with a lot of detail in them
    - **Solution**
      - Use an approximation of the model

E.g.,

Verify if the mouse is over the target

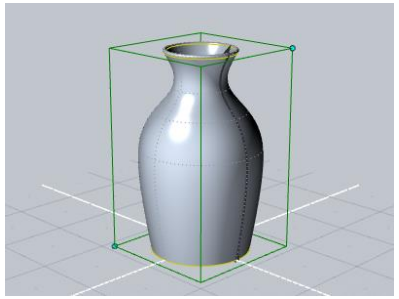
- Use the position of the target and the position of the mouse!

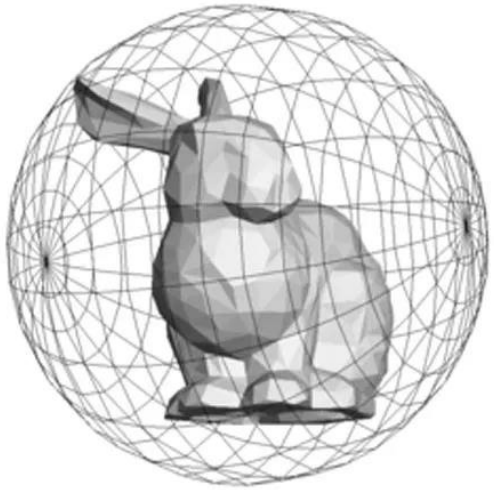


If Radius < Distance to Mouse
  Hit the target
Otherwise
  Miss the target

Approximation of models

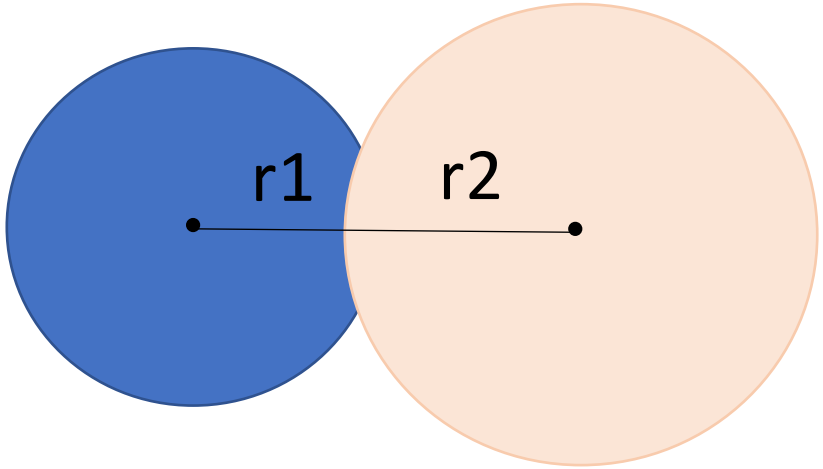
- The two methods :
  - bounding boxes
  - bounding spheres
- There will be area that is outside of the model but still inside of the bounding box or sphere
  - An ideal bounding region will limit this as much as possible.





Bounding Sphere

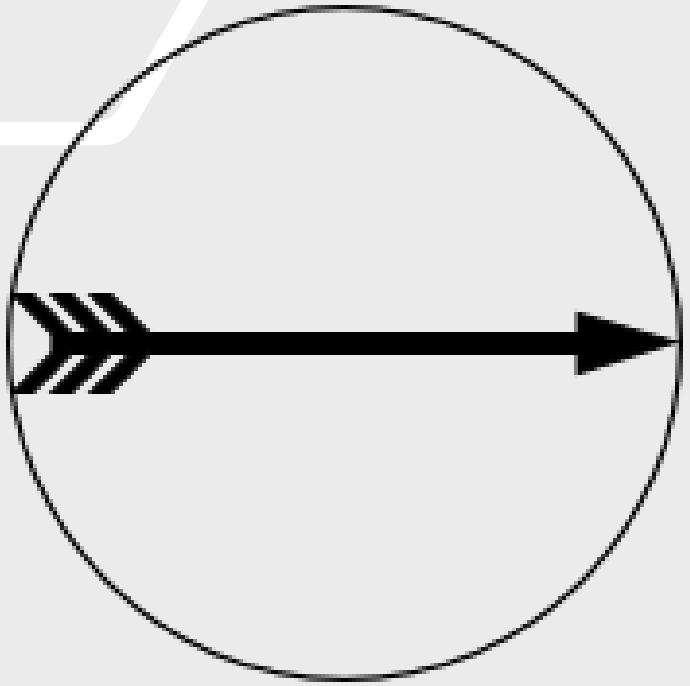
- For each model or for each mesh in a model
  - Construct a sphere from it.
  - The middle point of the model/mesh will become the center point for the bounding sphere
  - Figure out the farthest vertex in the model or mesh from the center point
  - The distance to from the farthest vertex to the center point is the radius of the bounding sphere
  - If the distance between two center points is less than the radii of the two bounding spheres, then there is a collision.



Distance < r1+r2

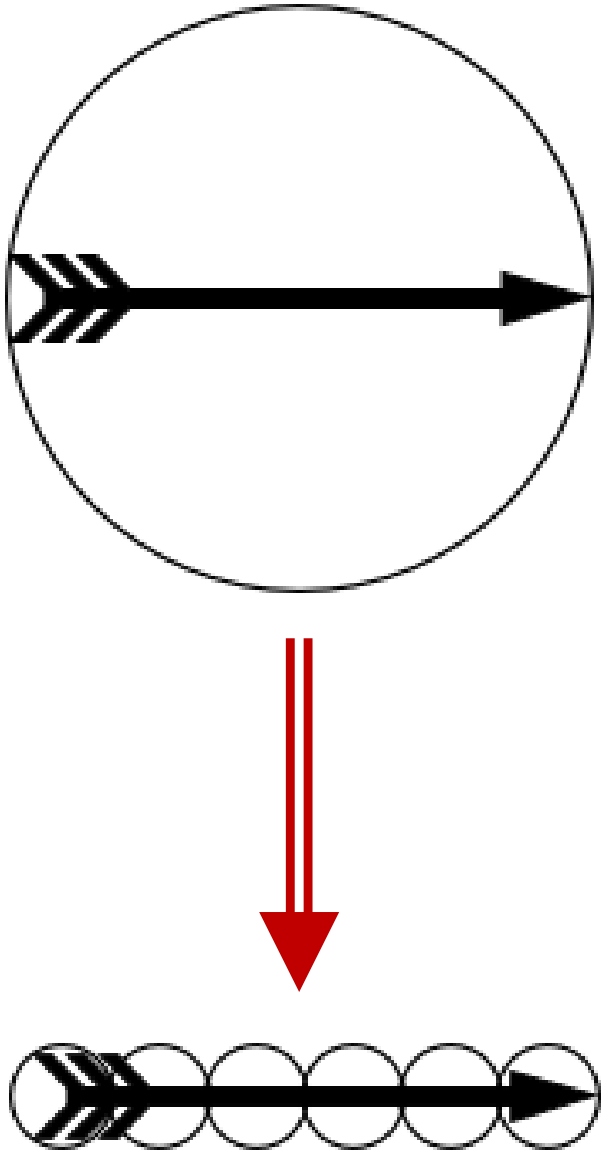
Drawback

- A sphere (or a bounding box) may not be a good approximation
  - E.g., if we have a long object
    - A bounding sphere might be a very bad approximation
    - It needs to be large enough to cover the entire object, but this allows a lot of extra space inside of the bounding sphere.



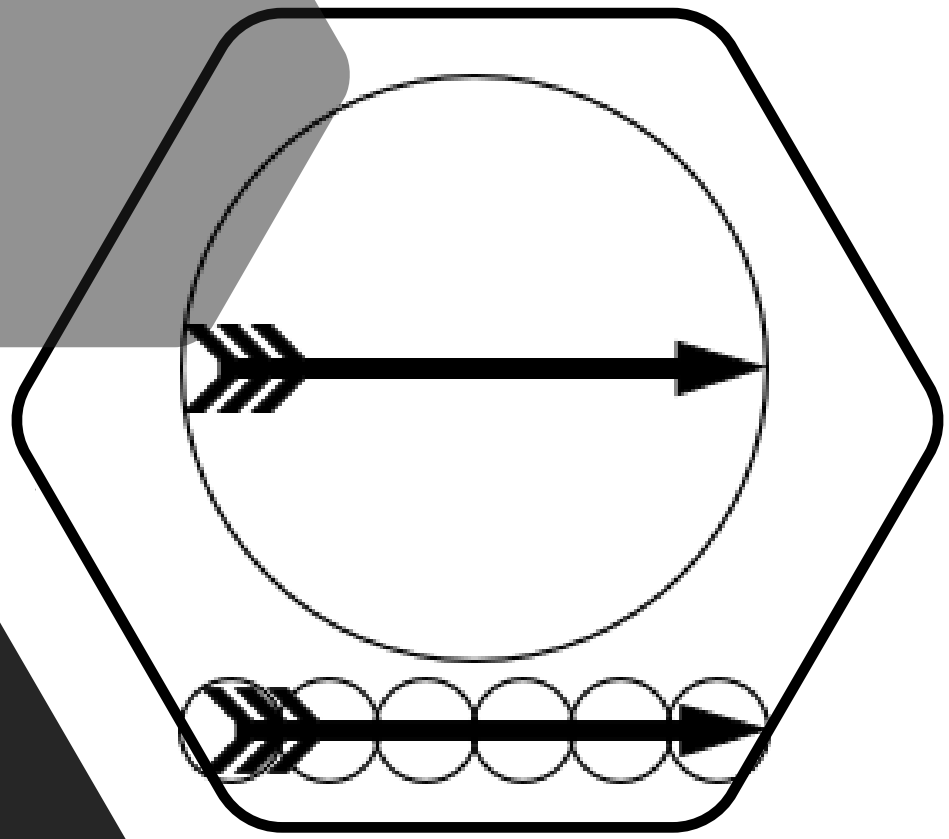
Solution

- Approximate the object with multiple spheres, rather than one
  - Allows for a better approximation of the object
  - **Drawback**
    - Comparing with many bounding spheres
    - May be inefficient if the other object is far away



Solution -- Hierarchical Model

- Use both the large sphere, and the small spheres
  - Compare with the big bounding sphere first
    - no collisions, continue with the game
    - collision, compare with the more detailed spheres



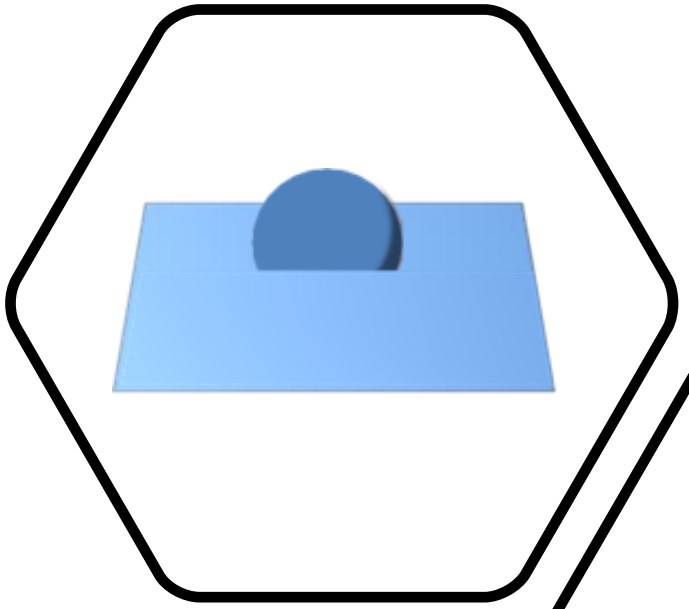
Solution -- Multiple Types of Approximation

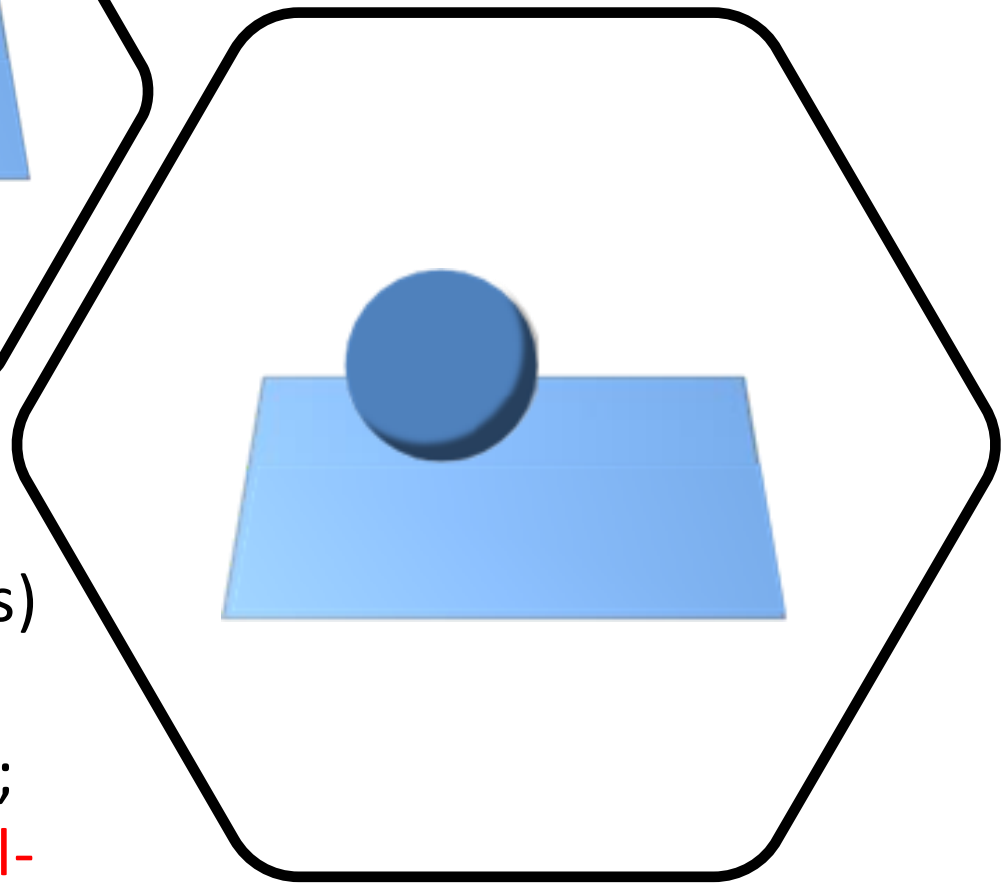
- We can combine the bounding spheres with bounding boxes.
- For some objects, bounding spheres are a better approximation of the object.
- For others, bounding boxes are. A system that allows either will be more accurate than one that strictly uses one or the other.

Detect collisions are straightforward, but fix collisions may not be…

E.g., sphere-plane collision on the right

Sphere-Plane Collision

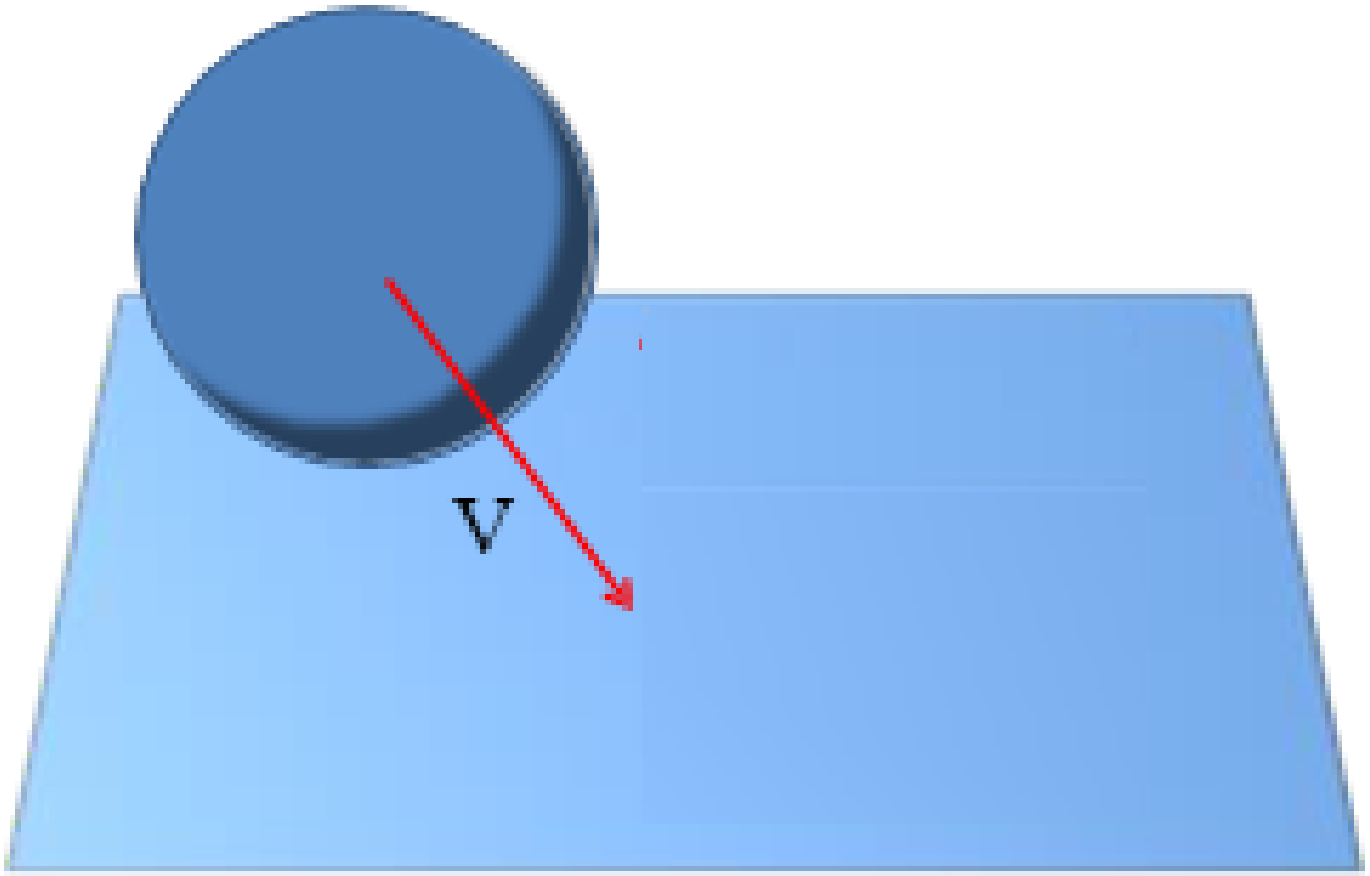




- Assume plane.Y=-20
- Detect the collision
  - sphere.center.Y < (-20.0f + sphere.radius)
- Fix the collision
  - sphere.center.Y = -20.0f + sphere.radius;
    - This may or may not work in the real-world

What if the Sphere is moving like this?

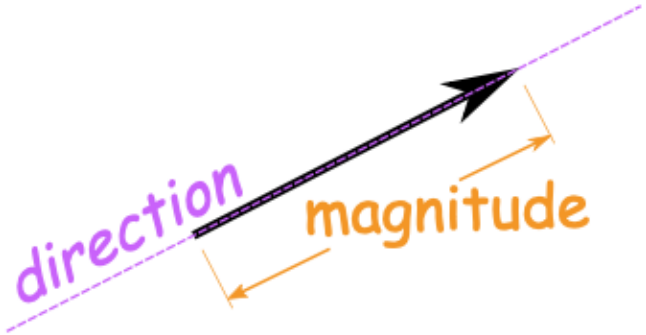
sphere.center.Y = -20.0f + sphere.radius **may not work!**



V is the direction vector

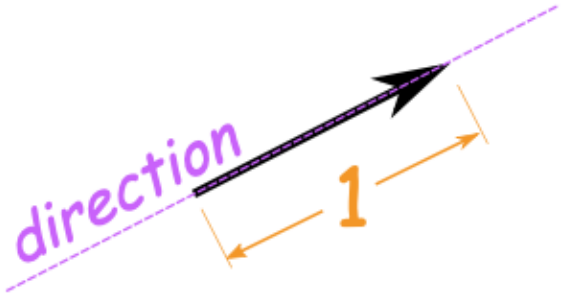
(A unit vector being used to represent spatial direction)

A vector has **magnitude** (how long it is) and **direction**:

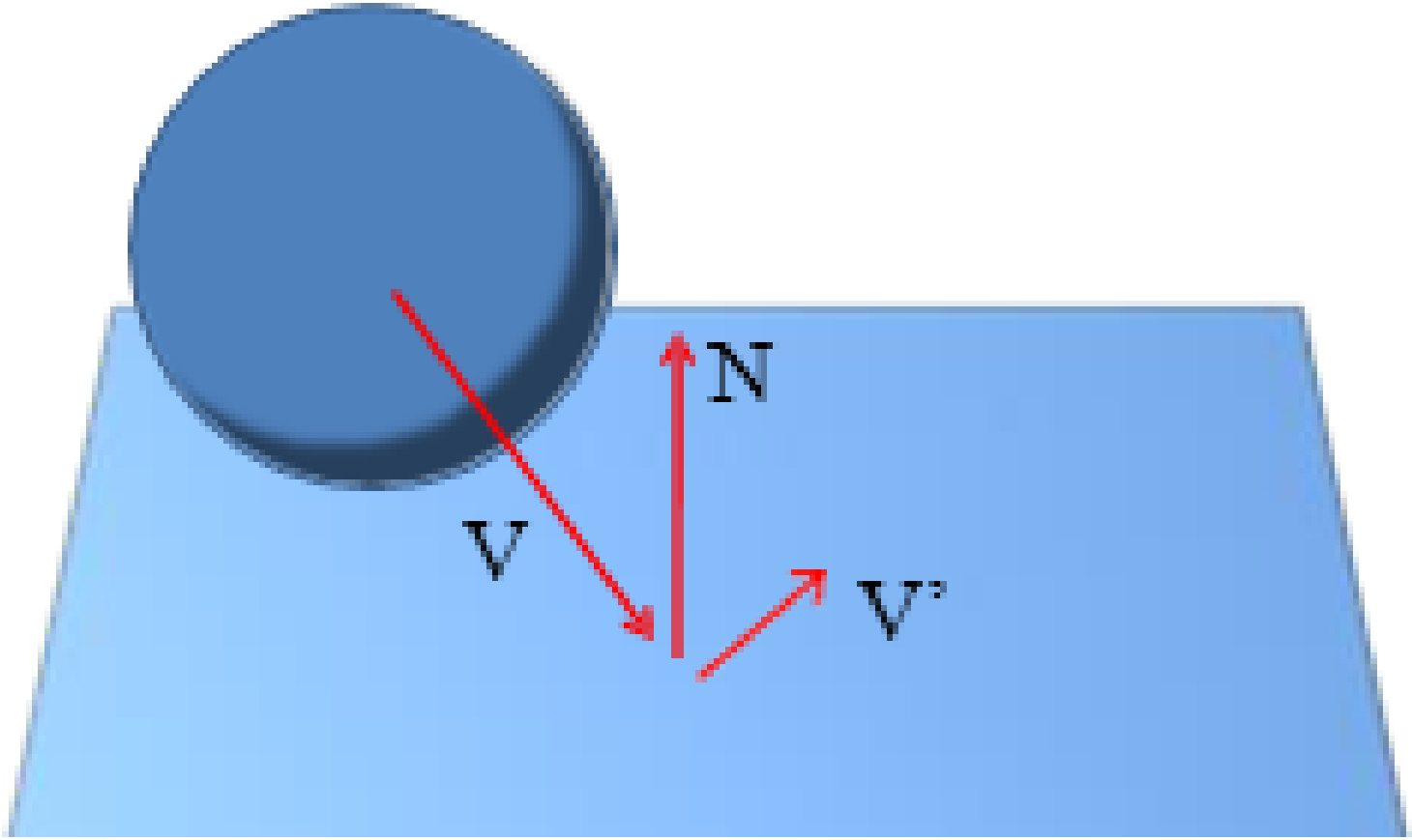


Unit Vector

A **Unit Vector** has a magnitude of **1**:



https://www.mathsisfun.com/algebra/vector-unit.html



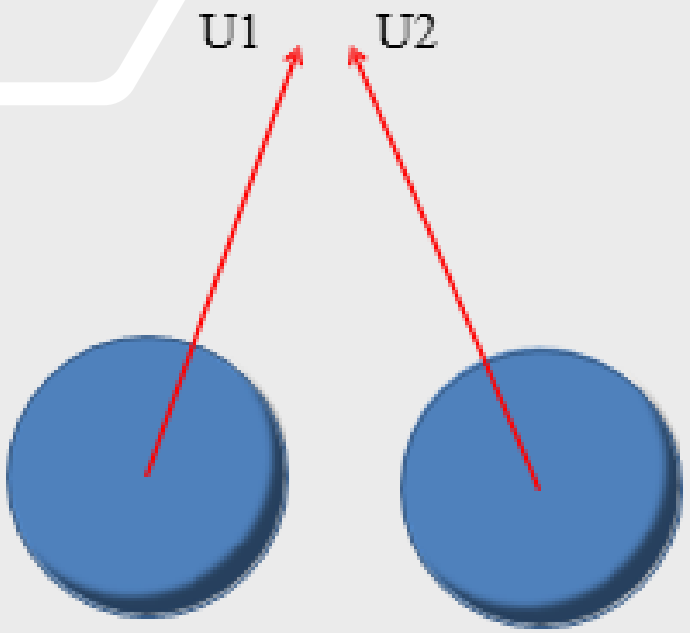
$$V' = 2 * (-V \text{ dot } N) * N + V$$

- V': the new direction vector
- V: the old direction vector before the collision
- N is the Normal at the collision point
  - "Up"
    - 0 1 0

```python
# ...

velocity = [0.01, -0.03, 0.0] # velocity of the moving object

def update(self):
        #...

        # --- 1. Apply velocity to position ---
        ball.data[0] += velocity1[0]
        ball.data[1] += velocity1[1]
        ball.data[2] += velocity1[2]

        # --- 2. Collision Detection (Hitting the "Floor") ---
        # Let's say our floor is at y = -0.8
        if self.ball.data[1] <= -0.8:
            # Prevent the object from sinking into the floor (fix position slightly)
            self.ball.data[1] = -0.8

            # Define vectors for Slide 14 math
            V = velocity1
            # Normal vector of a flat floor points straight UP
            N = [0.0, 1.0, 0.0]

            # --- 3. Collision Response (Slide 14 Formula) ---
            # Formula: V' = 2 * (-V dot N) * N + V

            # Calculate the dot product of (-V) and N
            minus_v_dot_n = sum(-v * n for v, n in zip(V, N))

            # Calculate the new velocity V'
            new_velocity = [0.0, 0.0, 0.0]
            for i in range(3):
                new_velocity[i] = 2 * minus_v_dot_n * N[i] + V[i]

            # Assign the new velocity
            velocity1 = new_velocity

        # Upload and draw the scene
        # ...
```

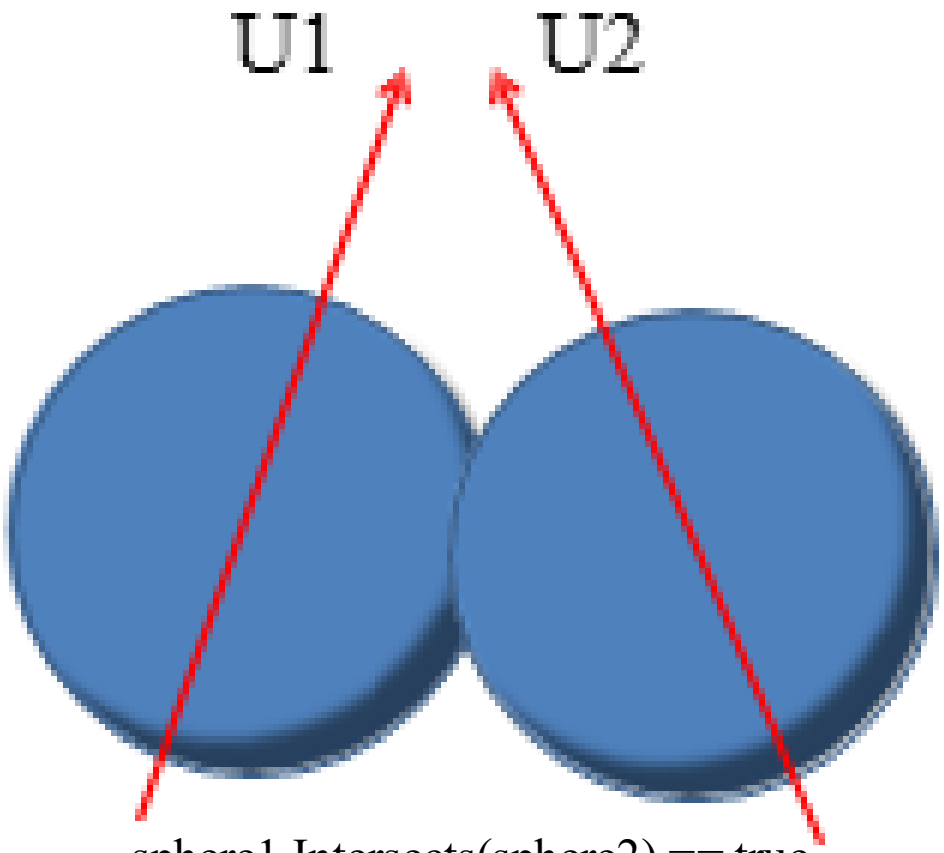
Two objects on a collision course

- Objects are almost colliding
- At their current velocity this will collide between frames
- This leaves us with an unrealistic rendering



Collision Has Occurred

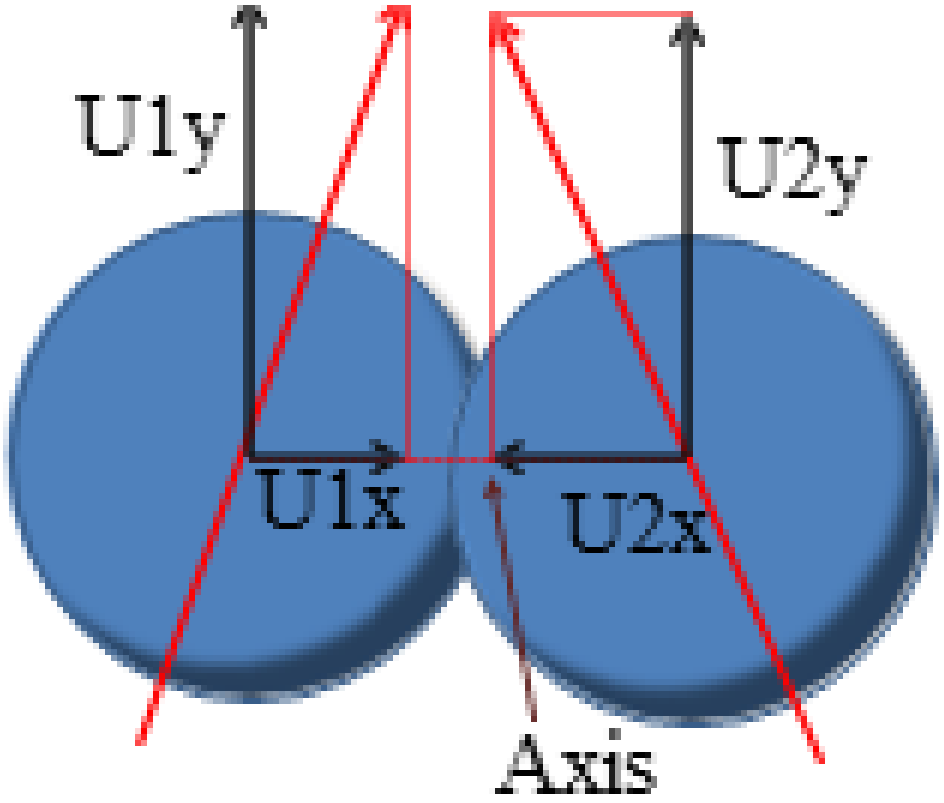
- Objects are intersecting
- Response
  - Calculate two things
    - Fix Directions
    - Fix Positions



sphere1.Intersects(sphere2) == true

Fix Direction – Method I

- Axis = C2 – C1
- U1x, U2x : projected vectors of the velocity vectors U1,U2 onto the Axis vector
  - U1x = Axis * (Axis dot U1)
  - U2x = -Axis * (-Axis dot U2)
- U1y and U2y : the projected vectors of the velocity vectors U1,U2 onto the axis which is perpendicular to Axis
  - U1y = U1 - U1x (remove the “collision” portion of the velocity)
  - U2y = U2 - U2x

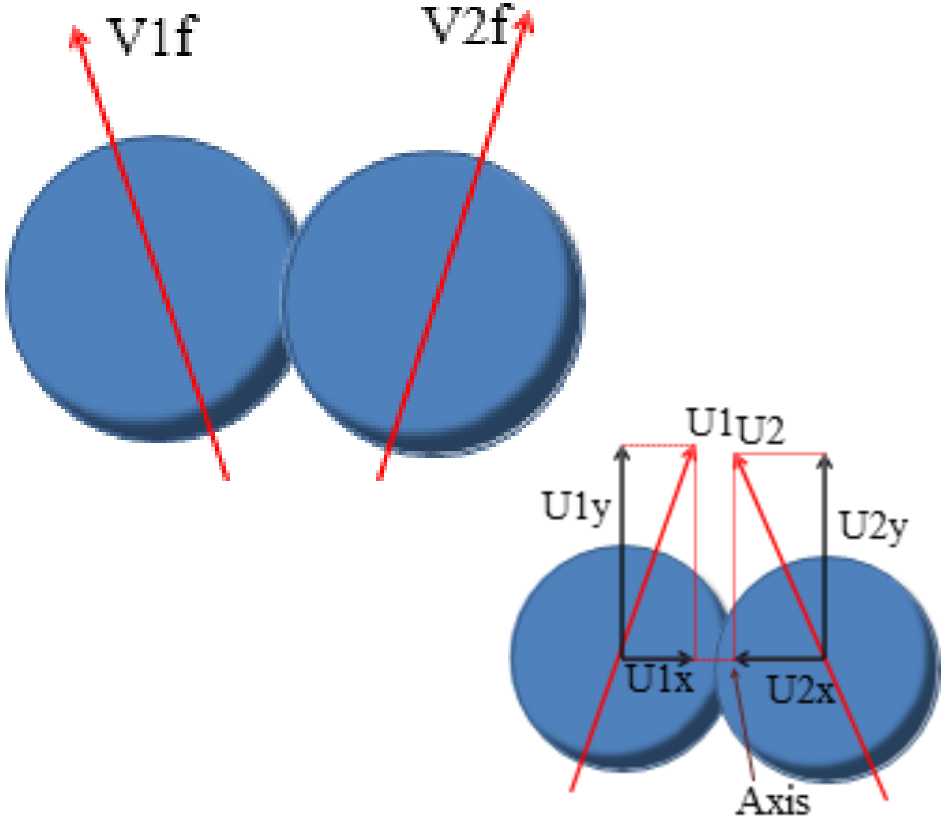


Fix Direction – Method II Elastic Collision

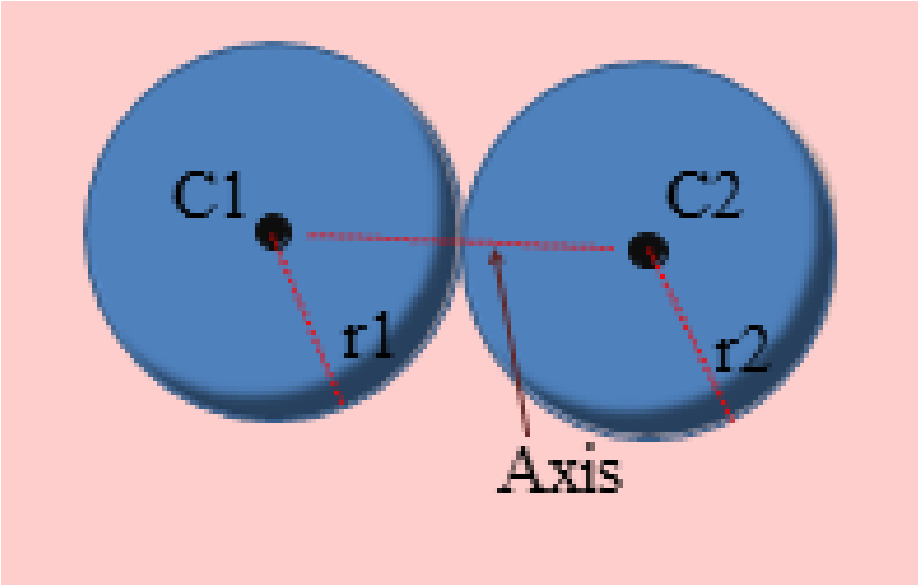
- Find the **velocity vectors**
- Velocity vectors represents the rate of change of the position of an object.
  - The magnitude of a velocity vector gives the speed of an object
  - The vector direction gives its direction

Elastic Collision (Cont)

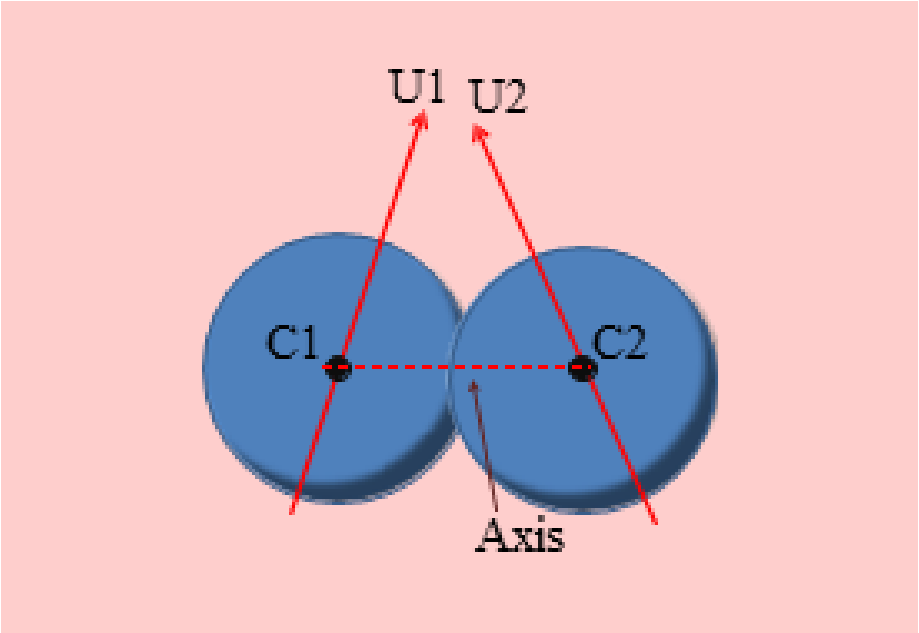
- Find the final velocity vector for each ball
  - V1f = V1x+V1y
  - V2f = V2x+V2y
- Elastic Collision Formula :
  - V1x = [(m1-m2)/(m1+m2)] U1x + [(2m2)/(m1+m2)] U2x
  - V2x = [(2m1)/(m1+m2)] U1x + [(m2-m1)/(m1+m2)] U2x
    - m: mass
  - V1y = U1y and V2y = U2y
    - Since the vectors U1y and U2y do not touch with each other, we know that the final velocity vector for these will be the same



Fix Position



- Vector3 Axis = C2 – C1
- float Dist = r1 + r2
- How far do we need to move each sphere?
  - Move = (Dist – Axis.length())/2
- How do we move it?
  - Axis.Normalize();
  - C1 = C1 - Axis*Move
  - C2 = C2 + Axis*Move



```
for (int i = 0; i < numSpheres; i++) {
  for (int j = 0; j < numSpheres; j++) {
    if (j != i) {
      if (spheres[i].Intersects(spheres[j])) {
        Vector3 axis = spherePos[j] - spherePos[i];
        float dist = spheres[i].Radius + spheres[j].Radius;
        float move = (dist - axis.Length()) / 2f;
        axis.Normalize();
        Vector3 U1x = axis * Vector3.Dot(axis, sphereVel[i]);
        Vector3 U1y = sphereVel[i] - U1x;

        Vector3 U2x = -axis * Vector3.Dot(-axis, sphereVel[j]);
        Vector3 U2y = sphereVel[j] - U2x;

        Vector3 V1x = ((mass[i] - mass[j]) / (mass[i] + mass[j])) * U1x
                    + ((2 * mass[j]) / (mass[i] + mass[j])) * U2x;
        Vector3 V2x = ((2 * mass[i]) / (mass[i] + mass[j])) * U1x
                    + ((mass[j] - mass[i]) / (mass[i] + mass[j])) * U2x;

        sphereVel[i] = (V1x + U1y);
        sphereVel[j] = (V2x + U2y);

        spherePos[j] += axis * move;
        spherePos[i] -= axis * move;
}}}}
```